CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In educational institutions, organizing and managing events such as technical seminars, workshops, cultural programs and competitions is an important activity. Traditionally, event management in colleges is handled manually, which can lead to issues such as miscommunication, delays in approval, difficulty in tracking student registrations, and inefficient record management.

To overcome these challenges, a College Event Management System is developed using the Salesforce platform. This system helps automate the entire event management process, starting from event creation to student registration. The administrator can create events, select the participating departments and assign faculty members responsible for organizing the events. Once an event is created, it is sent to the principal for approval through an automated approval process.

After the event is approved, the system automatically sends event details to students via email notifications. Students can view the event information and register for the event through the provided registration link. The system records all registrations and allows administrators and faculty members to monitor the number of participants and event details easily.

By using Salesforce features such as custom objects, flows, approval processes, Experience Cloud, reports, and dashboards, the system provides an efficient and centralized platform for managing college events. This helps reduce manual work, improves communication and ensures better organization of events within the institution.

1.2 EXISTING SYSTEM

In many colleges, event management is handled using manual methods such as paper forms, notice boards and email communication. Faculty members or administrators create event announcements and share the details with students through notices or messages. Students register for events by filling out paper forms or contacting faculty members directly.

This manual process can lead to several problems. Managing event details and student registrations becomes difficult when the number of participants increases. Tracking approvals from the principal may also take more time. There is no centralized system to store and manage event information, which can result in data loss or confusion.Because of these limitations, managing college events efficiently becomes challenging. Therefore, a digital system is needed to automate event creation, approval and student registration.

1.3 DRAWBACKS OF EXISTING SYSTEM

- The event management process is mostly manual and time consuming.
- Event information is shared through notices or messages, which may not reach all students.
- Tracking student registrations is difficult when the number of participants increases.
- There is no centralized system to store and manage event details.
- The approval process from the principal can take more time.
- Managing records using paper forms may lead to data loss or errors.
- It is difficult to generate reports or analyze event participation.

1.4 PROPOSED SYSTEM

The proposed system is a College Event Management System developed using Salesforce to automate and simplify the process of managing college events. In this system, the administrator creates events and selects the departments that are allowed to participate. The administrator also assigns a faculty member who will organize the event.After the event is created, it is sent to the principal for approval through an automated approval process. Once the principal approves the event, the system automatically sends the event details to students via email. The email contains the event information and a registration link.

Students can click the link to view the event details and register for the event. The system stores all registration details and automatically updates the number of participants for each event. Administrators and faculty members can monitor event registrations and generate reports and dashboards to analyze event participation.This system reduces manual work, improves communication and provides a centralized platform for managing college events efficiently.

1.5 ADVANTAGES OF PROPOSED SYSTEM

- The system automates the entire event management process.
- Event creation, approval and student registration become faster and easier.
- Event details are automatically sent to students through email notifications.
- Students can easily register for events using the registration link.
- All event and registration details are stored in a centralized system.
- The system reduces manual work and paperwork.
- Administrators and faculty can easily track student registrations.
- Reports and dashboards help analyze event participation.
- Communication between administrators, faculty and students is improved.

SUMMARY

The College Event Management System is developed to manage college events in a simple and efficient way. In the traditional method, events are managed manually using notices, emails or paper forms, which makes it difficult to track event details and student registrations. The proposed system uses Salesforce to automate the event process. The admin creates events and assigns faculty organizers and the event is sent to the principal for approval. After approval, event details are sent to students through email and students can register using the registration link. This system helps reduce manual work and improves event management in the college.

CHAPTER 2

SYSTEM ANALYSIS

2.1 IDENTIFICATION ON NEED

In many colleges, event management activities such as creating events, getting approvals and managing student registrations are often handled manually. Event information is usually shared through notice boards, emails or verbal communication. This manual process can cause problems such as delays in approval, difficulty in tracking student registrations and poor communication between faculty and students.

There is a need for a system that can manage the entire event process in a more organized and efficient way. The system should allow administrators to create events, assign faculty organizers and send the event for principal approval. After approval, the system should automatically notify students and allow them to register for events easily.

The College Event Management System is developed to solve these problems. It provides a centralized platform where event details, approvals and student registrations can be managed efficiently. This system improves communication, reduces manual work and helps the college manage events in a better and more structured manner.

2.2 FEASIBILITY STUDY

A feasibility study is an important part of system development. It helps to determine whether the proposed system can be designed, developed and implemented successfully. In this project, the feasibility study is used to evaluate whether the College Event Management System can work effectively in a college environment using the Salesforce platform. It also checks whether the system is practical, affordable and useful for the users.The feasibility of the proposed system can be explained under the following types.

2.2.1 Technical Feasibility

The proposed system is developed using the Salesforce platform, which provides tools such as custom objects, flows, approval processes and Experience Cloud. These technologies make it easy to build and manage the system. Since Salesforce is a cloud based platform, it does not require complex hardware or software installation.

2.2.2 Economic Feasibility

The system is cost effective because it reduces the need for manual work and paperwork. Using Salesforce helps automate many processes such as event creation, approval and student registration. This reduces administrative effort and improves efficiency.

2.2.3 Operational Feasibility

The system is easy to use for administrators, faculty members and students. Administrators can create events and manage registrations, faculty members can organize events and students can easily register through the registration link. This improves communication and simplifies event management in the college.

2.3 SYSTEM REQUIREMENTS SPECIFICATIONS

The College Event Management System requires both hardware and software resources for successful implementation. These requirements are listed below.

2.3.1 Hardware Requirements

- Computer or laptop
- Processor: Intel i3 or above
- RAM: 4 GB or above
- Hard Disk: 500 GB or above
- Internet connection

2.3.2 Software Requirements

- Operating System: Windows 10 or above
- Platform: Salesforce
- Browser: Google Chrome / Mozilla Firefox / Microsoft Edge

- Salesforce Tools:Custom Objects,Flows,Approval Process,Experience Cloud, Reports and Dashboards
- Programming/Development Tools:Visual Studio Code,Salesforce CLI,Lightning Web Components (LWC)

2.4 SOFTWARE DESCRIPTION

2.4.1 FRONT END

HTML & CSS

HTML (Hypertext Markup Language) and CSS (Cascading Style Sheets) are essential tools for building websites, each with its strengths and limitations. HTML is easy to learn, making it beginner friendly and since it's supported by all browsers, it's widely accessible. It’s also free to use and helps with SEO through semantic tags, which make content easier for search engines to understand. It can also get repetitive, especially in larger projects and without additional technologies like CSS or JavaScript, web pages remain static and basic. In short, HTML provides the structure and CSS handles the appearance, working together to create modern, functional and visually appealing websites. However, both require careful management, especially in larger projects, to avoid issues and ensure the final product is both effective and easy to maintain.

2.4.2 SCRIPT LANGUAGE

JAVASCRIPT

JavaScript is a versatile, high level programming language widely used for creating dynamic and interactive web content. It is one of the core technologies of the web, alongside HTML and CSS and primarily runs on the client side within web browsers. JavaScript allows developers to build interactive user interfaces, validate forms, update content dynamically without reloading the page and add animations to enhance user experience.

It is event driven, lightweight and features dynamic typing, making it highly flexible for various applications. Beyond frontend development, JavaScript can also be used on the back end with frameworks like Node.js, enabling fullstack development. Its crossplatform compatibility, vast ecosystem of libraries and frameworks and ease of

learning make it a popular choice among developers. However, care must be taken to address potential security issues, as JavaScript code runs directly in the browser.

2.4.3 BACK END

SALESFORCE FLOWS

Salesforce Flow is an automation tool used to perform actions automatically without writing code. It helps automate business processes by creating a sequence of steps that the system follows.Flows can perform tasks such as creating records, updating records, sending emails or displaying forms to users. They work based on conditions and triggers, which means the flow starts when a specific event occurs.

There are different types of flows, such as Screen Flow, Record Triggered Flow and Auto launched Flow. These flows help organizations reduce manual work and improve efficiency by automating repetitive tasks.

APPROVAL PROCESS

An Approval Process is used when a record needs permission or validation from a higher authority before it becomes final. It helps ensure that important actions are reviewed and approved by authorized users.

In Salesforce, when a record is submitted for approval, it is automatically sent to the designated approver. The approver can either approve or reject the request. Based on the decision, the system updates the record status and may perform additional actions such as sending notifications or updating fields.Approval processes help maintain control, accuracy and accountability within an organization.

TRIGGERS

A Trigger in Salesforce is a piece of Apex code that runs automatically when certain actions happen on records in the database. Triggers execute before or after operations such as inserting, updating or deleting records.

Triggers are used to perform complex logic that cannot be handled by standard automation tools. For example, they can validate data, update related records or enforce

business rules.Triggers help maintain data integrity and ensure that specific actions are automatically executed when changes occur in the system.

2.4.4 DATABASE

SALESFORCE CLOUD DATABASE

The Salesforce Cloud Database is used to store and manage all the data in the system. It is a cloud based database provided by the Salesforce platform. In Salesforce, users do not directly manage the database like traditional databases such as MySQL or Oracle. Instead, Salesforce automatically manages data storage, security, and performance.The Salesforce database stores data using objects, fields and records. Objects are similar to tables in a traditional database, fields are similar to columns and records represent the actual data stored in the system. This structure helps organize data in a clear and efficient way.

The Salesforce Cloud Database also supports relationships between different objects, which helps connect related information easily. For example, event details, user information and registration data can be linked together within the system.Since it is a cloud based database, data can be accessed securely from anywhere through the internet. Salesforce also provides strong security features to protect the stored data. This makes the Salesforce Cloud Database reliable, scalable and suitable for managing application data efficiently.

SUMMARY

This chapter explains the system analysis of the College Event Management System. It identifies the need for a digital system to manage events because manual methods make event management difficult. The feasibility study shows that the system can be developed using Salesforce and is easy to use and cost effective. The chapter also explains the hardware and software requirements needed for the system. The front end uses HTML and CSS, JavaScript is used as the scripting language and the backend uses Salesforce Flows, Approval Process and Triggers. All data is stored in the Salesforce Cloud Database, which helps manage event and registration information securely.

CHAPTER 3

SYSTEM DESIGN

3.1 MODULE DESCRIPTION

The College Event Management System is divided into different modules based on the functions performed by each user. Each module handles a specific task in the system. The main modules of the system are described below.

3.1.1 Admin Module

The Admin Module is responsible for creating and managing events. The admin enters event details such as event name, date, venue and event type. The admin also selects the departments that are allowed to participate in the event and assigns the faculty member who will organize it. After creating the event, the admin sends it to the principal for approval.

3.1.2 Principal Approval Module

The Principal Approval Module is used to review the event created by the admin. The principal can check the event details and either approve or reject the event. If the event is approved, it moves to the next stage. If it is rejected, the admin can make changes and send it again for approval.

3.1.3 Email Notification Module

The Email Notification Module automatically sends event details to students after the event is approved. The email contains information such as event name, date, venue and a registration link. This helps students receive the event information quickly and easily.

3.1.4 Student Registration Module

The Student Registration Module allows students to register for approved events. Students can click the registration link received in the email, view the event details and submit their registration. The system stores the student registration details in the database.

3.1.5 Registration Management Module

The Registration Management Module keeps track of all student registrations for each event. It updates the registration count whenever a student registers. This module helps the admin and faculty members monitor the number of participants.

3.1.6 Report and Dashboard Module

The Report and Dashboard Module is used to generate reports and display charts related to events and student registrations. It helps in analyzing approved events, registration count, department wise participation, gender-wise participation and event type statistics.

3.2 DATA FLOW DIAGRAM

A Data Flow Diagram is a graphical representation that shows how data moves through a system. It explains how information flows between different entities, how the data is processed and where the data is stored. DFD helps in understanding the system structure and the interaction between users and the system.

3.2.1 DFD Level 0

DFD Level 0 provides a high level overview of the system by representing the entire system as a single process and showing how external entities interact with it. It focuses only on the main data flow and does not include detailed internal processes.In the College Event Management System, the main external entities are Student, Faculty, Principal and Admin. All interactions occur through the central College Event Management System.

The Student interacts with the system by viewing event details and registering for events. The system provides event information and confirms the registration.The Faculty

interacts with the system by managing events and organizing event activities. Faculty members can view event details and manage the events assigned to them.

The Principal interacts with the system by reviewing event requests and approving or rejecting them. The approval process ensures that only authorized events are conducted in the college.The Admin manages the overall system by creating event profiles, assigning faculty organizers and monitoring event activities. The admin can also view system reports and event details.

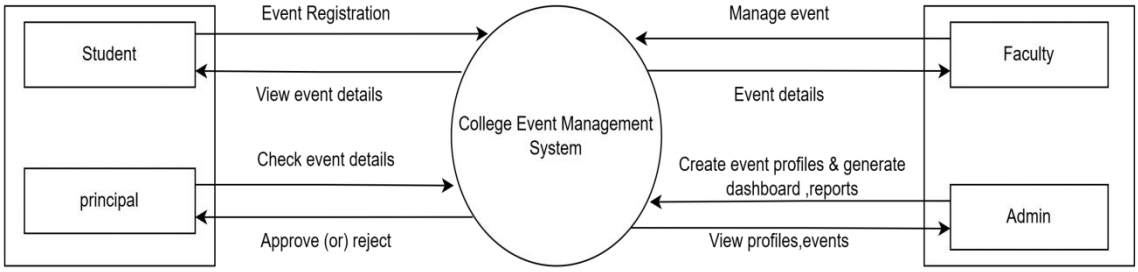


Fig 3.1 Data Flow Diagram Level 0

Figure 3.1 illustrates the Level 0 Data Flow Diagram of the College Event Management System, showing how data flows between external entities such as Student, Faculty, Principal and Admin and the central system. The diagram highlights the main interactions and defines the system boundary clearly.

3.2.2 DFD Level 1

The DFD Level 1 diagram explains the internal functioning of the College Event Management System by dividing the system into major processes and illustrating how data flows between users, system modules and databases. This level provides a more detailed representation of the system compared to the context diagram by showing the interaction between different processes such as User Registration & Login, Event Creation & Management, Event Approval Process, Student Event Registration and Report & Dashboard Management. Each process exchanges data with corresponding databases to ensure smooth execution of event management activities.

User Registration & Login

The User Registration & Login process allows authorized users such as the Admin, Faculty and Principal to access the system. When a user logs into the system, the entered credentials such as username and password are verified with the information stored in the User Database. If the credentials are valid, the system grants access based on the user's role. This process ensures that only authorized users can access the system and perform specific operations, thereby maintaining security and proper access control.

Event Creation & Management

Event Creation & Management process is mainly handled by the admin. In this process, the admin enters important event details including the event name, event date, venue, participating departments and the faculty organizer responsible for conducting the event. Once the details are entered, the system stores the information in the Event Database. The assigned faculty member can view and manage the event activities through the system. This process ensures that all event related information is properly recorded and maintained.

Event Approval Process

The Event Approval Process ensures that every event created by the admin is reviewed by the principal before it becomes available to students. After the admin creates the event, the system sends the event details to the principal for approval. The principal reviews the event information and decides whether to approve or reject it. If the principal approves the event, the system updates the event status in the Event Database and allows the event to move to the next stage. Once approved, the system automatically sends event notifications to students informing them about the event details and registration options.

Student Event Registration

The Student Event Registration process enables students to register for approved events. Students can view event details through the event notification email or registration link. After selecting the event, the student fills out the registration form with required information such as name, department, register number and contact details. The system then stores the submitted information in the Registration Database. After successful

registration, the system updates the registration count and confirms the student's participation.

Reports & Dashboard Management

The Reports & Dashboard Management process helps administrators analyze event activities and participation using reports and dashboards. In this process, the admin can generate various reports related to events and student registrations. The system collects data from the Event Database and Registration Database to generate analytical reports and visual dashboards. These reports may include information such as total events conducted, department wise participation, gender wise participation, and total registrations. This module helps the administration monitor event performance and make informed decisions.

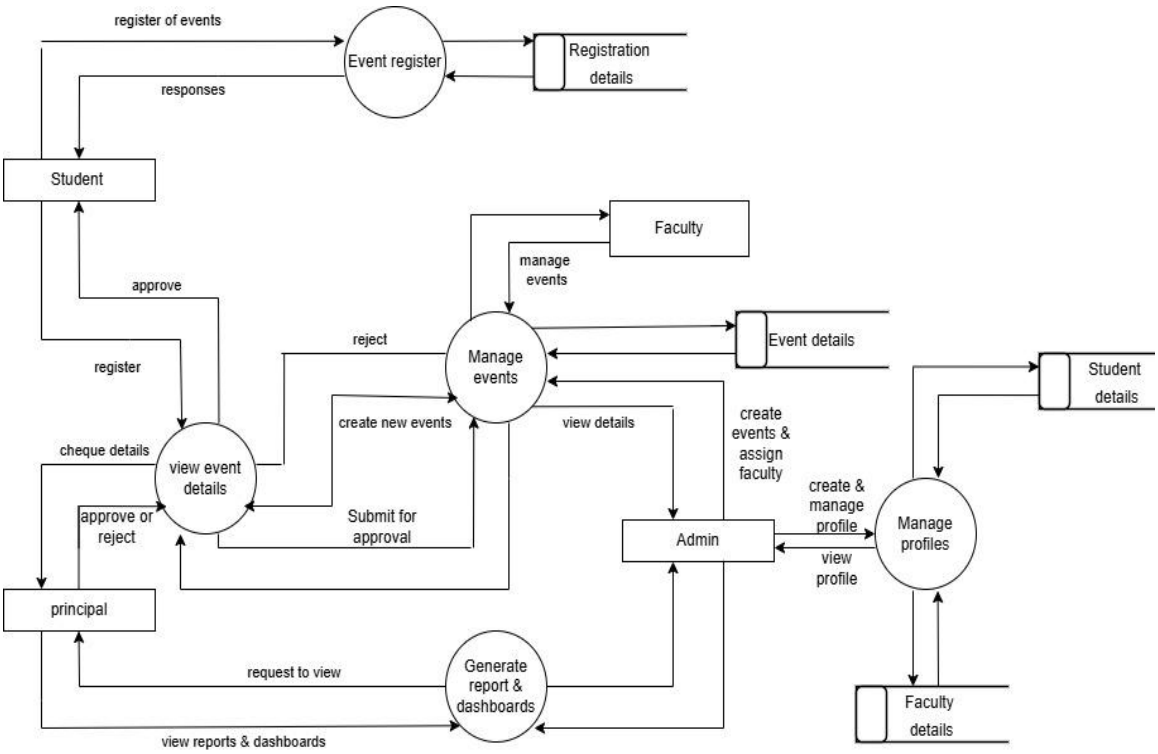


Fig 3.2 Data Flow Diagram Level 1

Figure 3.2 illustrates the Level 1 Data Flow Diagram of the College Event Management System, showing detailed internal processes such as event registration, event management, event approval and profile management. The diagram also shows how data flows between users, processes and system databases.

3.3 USE CASE DIAGRAM

A Use Case Diagram represents the functional requirements of a system by showing the interaction between users (actors) and the system. It explains what actions different users can perform within the system. Use Case Diagrams help in understanding the system behavior from the user's perspective and clearly define the system functionality.The Use Case Diagram of the College Event Management System identifies four main actors: **S**tudent, Faculty, Principal and Admin. Each actor interacts with the system based on their roles and responsibilities.

Student

The Student can view event details and register for events. Students receive information about approved events and can participate by submitting their registration through the system. Students can also view the event date, venue and other details before registering. The system records the student registration information and updates the event participation count.

Faculty

The Faculty is responsible for managing the events assigned to them. Faculty members can view event details, manage event activities and monitor event information through the dashboard.Faculty members help in organizing and coordinating the event activities. They can also monitor the number of students registered for the event.

Principal

The Principal reviews the events created by the admin. The principal checks the event details and either approves or rejects the event. This ensures that only authorized events are conducted in the college.

Admin

The Admin manages the overall system. The admin can create events, manage user profiles, generate dashboards and reports and delete or update event details. The admin is responsible for maintaining the entire event management process.

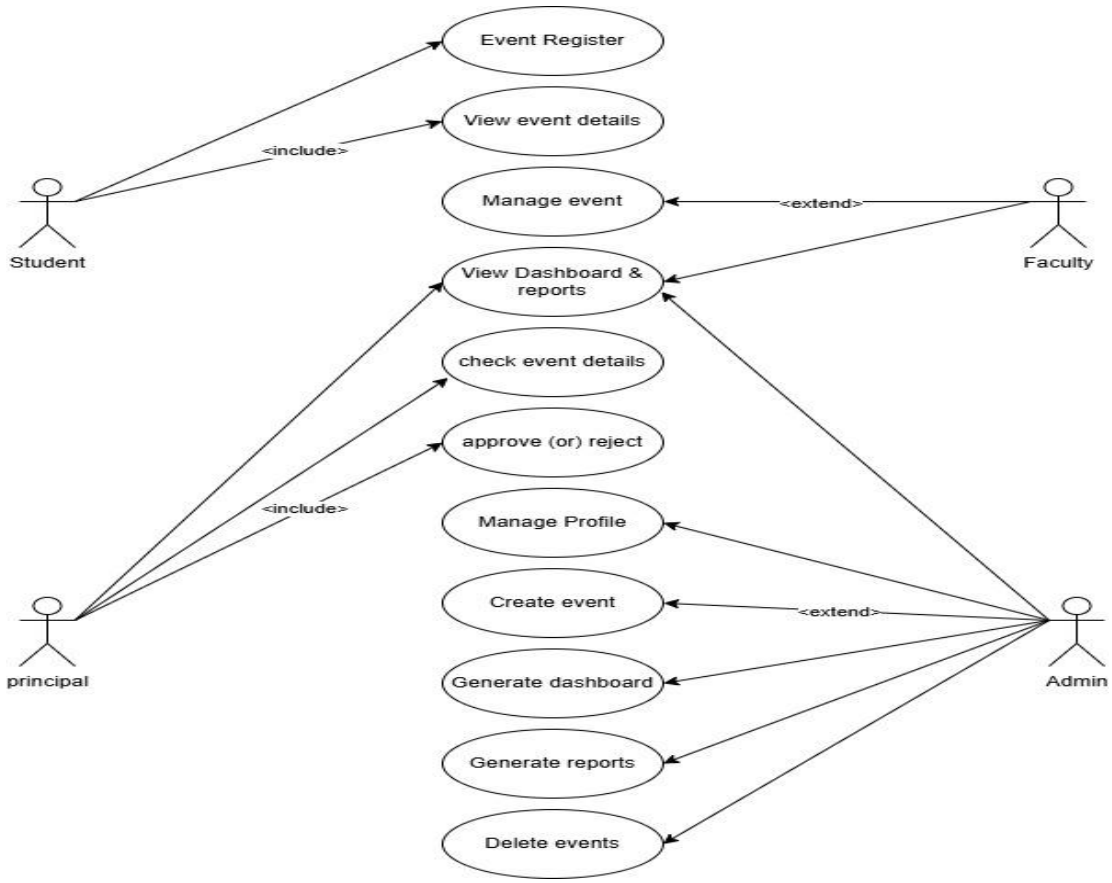


Figure 3.3 Use Case Diagram

Figure 3.3 illustrates the Use Case Diagram of the College Event Management System. It shows the interaction between different actors and the system functions. The diagram provides a clear overview of user roles and the operations performed in the system.

3.4 INPUT DESIGN

Input design involves creating user interfaces that allow users to enter data into the system accurately and efficiently. A good input design helps reduce errors, improves usability and ensures smooth interaction between the user and the system. Proper input design focuses on simplicity, clarity and validation so that users can easily provide the required information.

In the College Event Management System, input forms are designed to support event creation, approval processes and student event registration.

Faculty Login Form

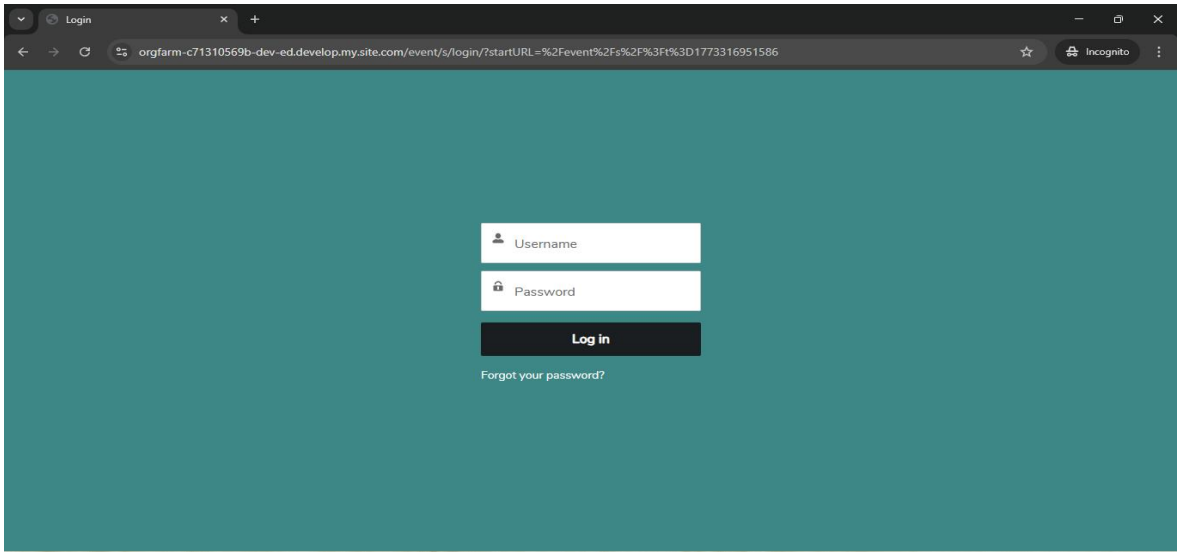


Figure 3.4 Faculty Login Form

Figure 3.4 illustrates the Faculty Login Form, which is used by faculty members to securely access the College Event Management System. This form acts as the authentication interface where faculty users must enter their registered username and password to log into the system. The login form typically contains input fields for the username and password along with a login button that submits the credentials for verification.

When the faculty member enters the login details and submits the form, the system validates the credentials by comparing them with the stored information in the user database. If the entered username and password match the records in the database, the faculty member is successfully authenticated and granted access to the system. If the credentials are incorrect, the system displays an error message and requests the user to reenter the correct login information.

After successful authentication, the faculty member is redirected to the Faculty Dashboard, where various event management functions are available. From this dashboard, the faculty member can view event details, monitor student registrations and manage event related information. Faculty members can check the list of students who have registered for events, track participation details and coordinate the event activities assigned to them.

Event Creation Form

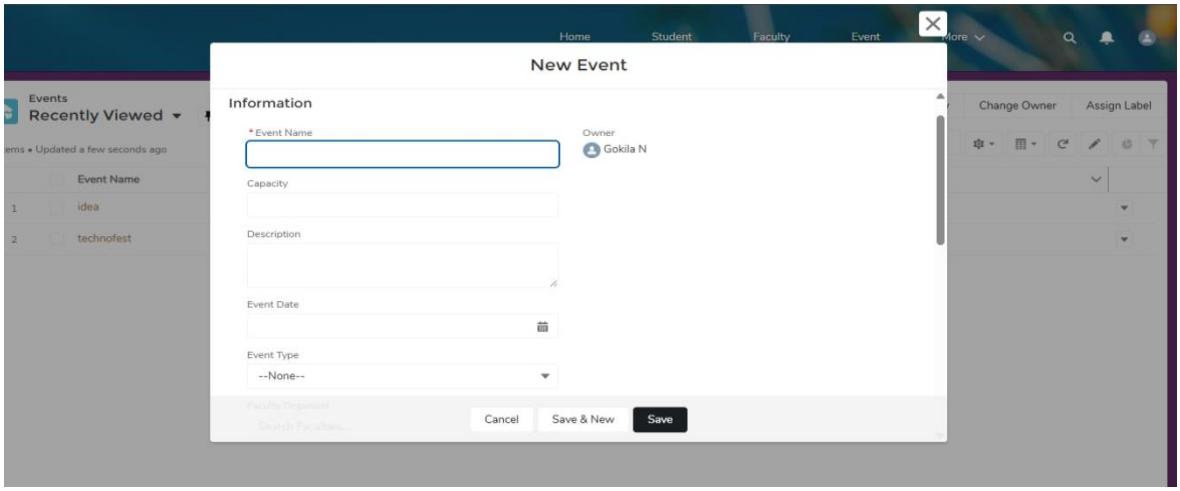


Figure 3.5 Event Creation Form

Figure 3.5 illustrates the Event Creation Form, which is used by the administrator to create and add a new event in the College Event Management System. This form allows the user to enter all necessary information related to the event so that it can be properly recorded and managed within the system.

The form collects important event details such as the Event Name, which identifies the event and the Event Type, which specifies whether the event is Technical or Non Technical. The user must also enter the Event Date and Venue, which inform students about when and where the event will be conducted. In addition, the form includes a Registration Deadline field that indicates the last date for students to register for the event.

The administrator or faculty member can also provide a detailed event description, which explains the purpose, rules or additional information related to the event. The form also allows the user to upload an Event Flyer, which visually represents the event and provides additional promotional information to students.

After entering all the required details, the user submits the form and the event information is stored in the event database of the system. Once the event is successfully created, it is automatically forwarded to the principal for approval. The event will only become visible to students after the principal reviews and approves it. This process ensures that all events are properly verified before they are published in the system.

Student Registration Form

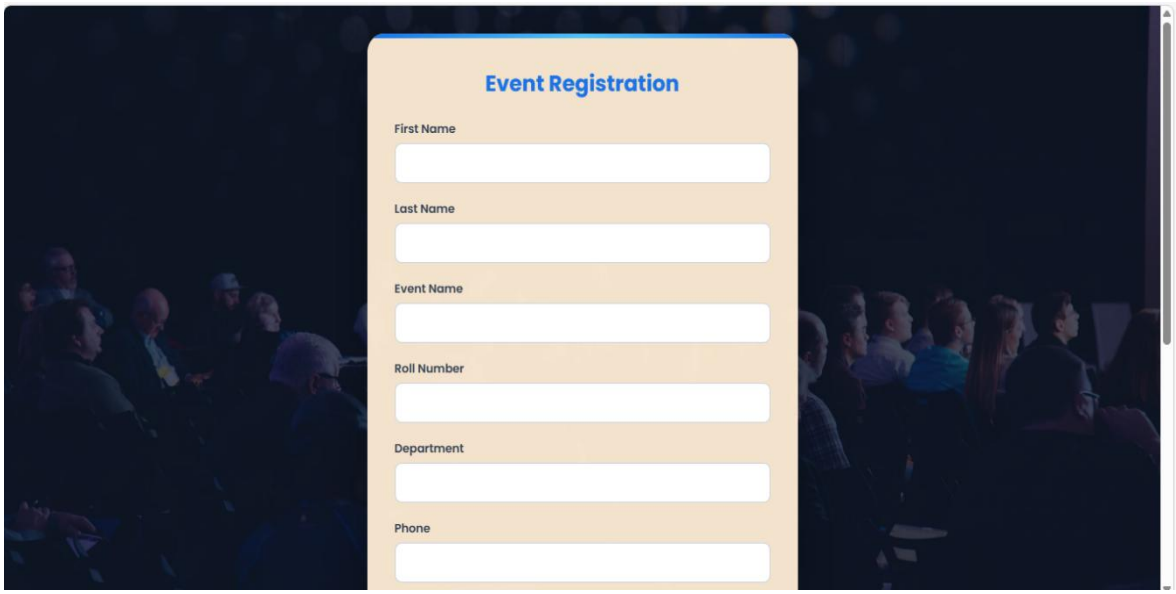


Figure 3.6 Student Event Registration Form

Figure 3.6 illustrates the Student Event Registration Form, which is used by students to register for events in the College Event Management System. Students receive an email notification containing the event details and a registration link. By clicking the registration link, students are directed to the registration form where they can submit their information to participate in the event.

The registration form collects important student details such as Student Name, Email Address, Department and Phone Number. These details help the system identify the participant and maintain proper records of event registrations. Students are required to fill in all the necessary fields before submitting the form.

Once the student submits the registration form, the system stores the registration information in the registration database. The system also automatically updates the event registration count, which helps administrators and faculty members monitor the number of participants for each event.

This input design makes the registration process simple and user friendly for students while ensuring that all participant details are accurately recorded and managed within the system.

3.5 OUTPUT DESIGN

Output design refers to how processed information is presented to users in a clear and meaningful format. A well designed output screen helps users understand the system information quickly and make better decisions.

In the College Event Management System, output screens are designed to display event details, student registrations and analytical reports.

Admin Dashboard

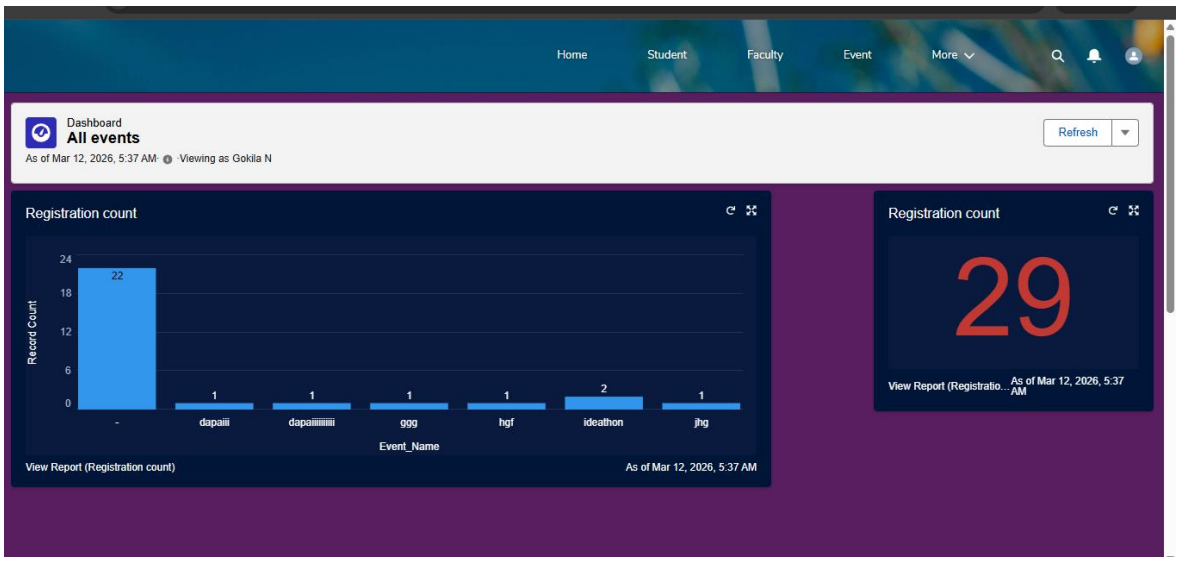


Figure 3.7 Admin Dashboard

Figure 3.7 illustrates the Admin Dashboard, which serves as the central control panel of the College Event Management System. The dashboard provides the administrator with a complete overview of the system and helps in monitoring and managing all event related activities effectively.

The dashboard displays important summary information such as Total Events Created, Total Approved Events, Total Student Registrations and Upcoming Events. These details help the administrator quickly understand the current status of events and student participation within the system.

From the dashboard, the administrator can also access different system modules such as Event Management, Faculty Management, Student Registration Details and Reports and Dashboards. These modules allow the admin to create and manage events,

view and maintain faculty information, monitor student registrations and analyze event data through reports and graphical dashboards.

The Admin Dashboard provides a user friendly interface that allows the administrator to efficiently manage the entire event management process. It helps in tracking event activities, monitoring participation statistics and making informed decisions for organizing future events.

Event Details Page

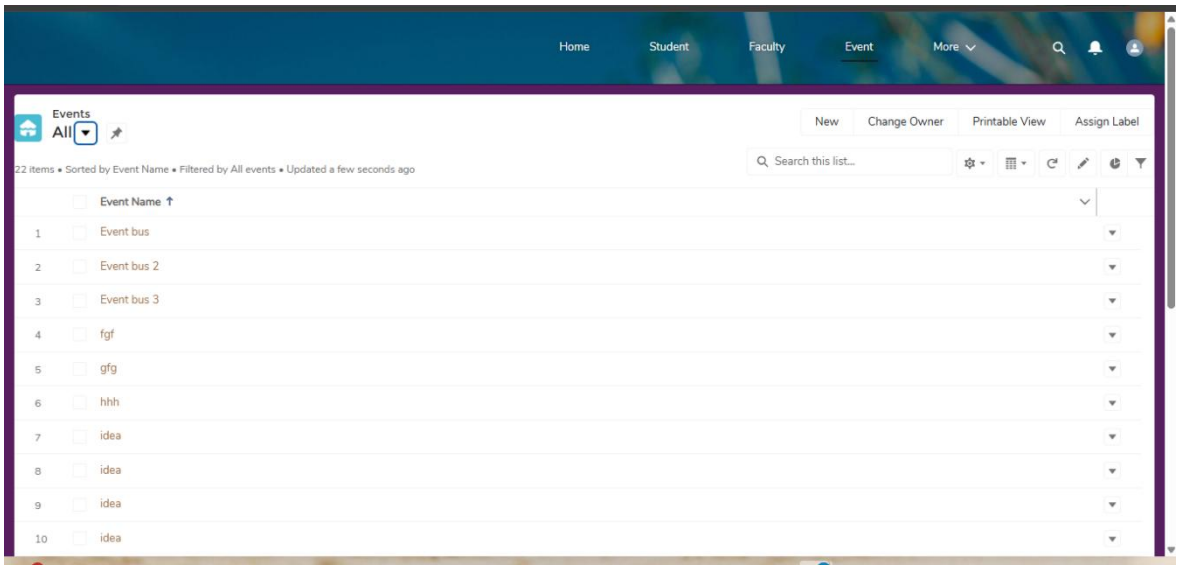


Figure 3.8 Event Details Page

Figure 3.8 shows the Event Details Page, which displays detailed information about each event in the College Event Management System. This page is mainly used by the administrator and faculty members to view and manage event information.

The page displays important event details such as Event Name, Event Date, Event Type, Venue, Event Status (Approved or Pending) and Registration Count. These details help the admin and faculty members monitor the event status and track the number of students registered for each event.

The Event Details Page helps administrators manage events efficiently and ensures that all event information is properly maintained in the system.

Event Registration Report

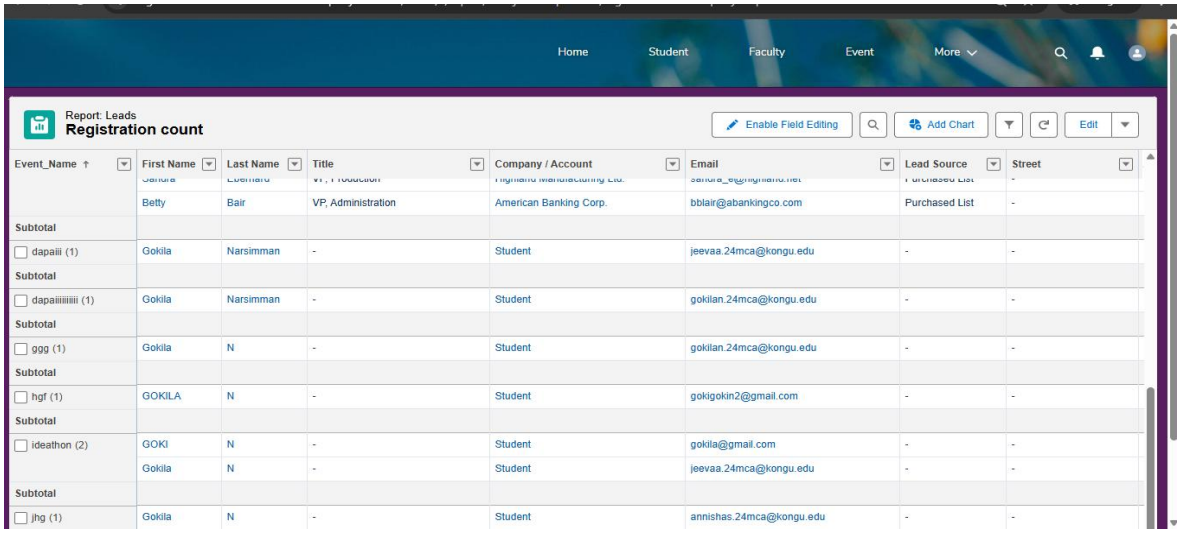


Figure 3.9 Event Registration Report

Figure 3.9 shows the Event Registration Report, which provides detailed information about the number of students registered for each event in the College Event Management System. This report helps administrators and faculty members easily monitor and analyze student participation in various events.

The report displays important information such as the event name, total number of registered students and participation details. By viewing this report, administrators and faculty members can track how many students have registered for a particular event and identify events with high or low participation.

This report also helps in tracking event participation, monitoring student registrations and analyzing the popularity of different events. In addition to tabular reports, the system can generate charts and dashboards to visually represent registration statistics. These visual outputs may include department wise registrations, gender wise registrations and participation comparison between technical and non technical events.

The Event Registration Report helps the administration understand event performance, evaluate student engagement and make better decisions when planning future events.

3.6 ER DIAGRAM

The Entity Relationship (ER) Diagram represents the database structure of the College Event Management System. It shows the main entities in the system and the relationships between them.

Event Object

The Event object stores all event-related information. It contains details such as event name, event date, venue, event type, status and registration count. This object is used to manage the complete details of each event created in the system.

Important fields in Event object:

- Event Name
- Event Date
- Venue
- Event Type
- Status
- Registration Count

Faculty Object

The Faculty object stores information about the faculty members who organize events. It contains details such as faculty name, email, phone number, department and designation.

Important fields in Faculty object:

- Faculty Name
- Faculty Email
- Faculty Phone
- Faculty Department
- Faculty Designation

Student Object

The Student object stores student details required for registration and participation. It contains information such as student name, email, gender, department and year of study.

Important fields in Student object:

- Student Name
- Student Email
- Gender
- Department
- Year

Registration Object

The Registration object stores the event registration details of students. It connects students with events and records who registered for which event. This object is used to maintain participation records.

Important fields in Registration object:

- Registration ID
- Event Name
- Student ID
- Registration Date

Relationships

The database also includes relationships between these objects:

- One Faculty can organize many Events
- One Event can have many Registrations
- One Student can register for many Events
- One Principal/Admin can approve multiple Events

These relationships help maintain data consistency and make it easy to track event creation, approval, and student participation.

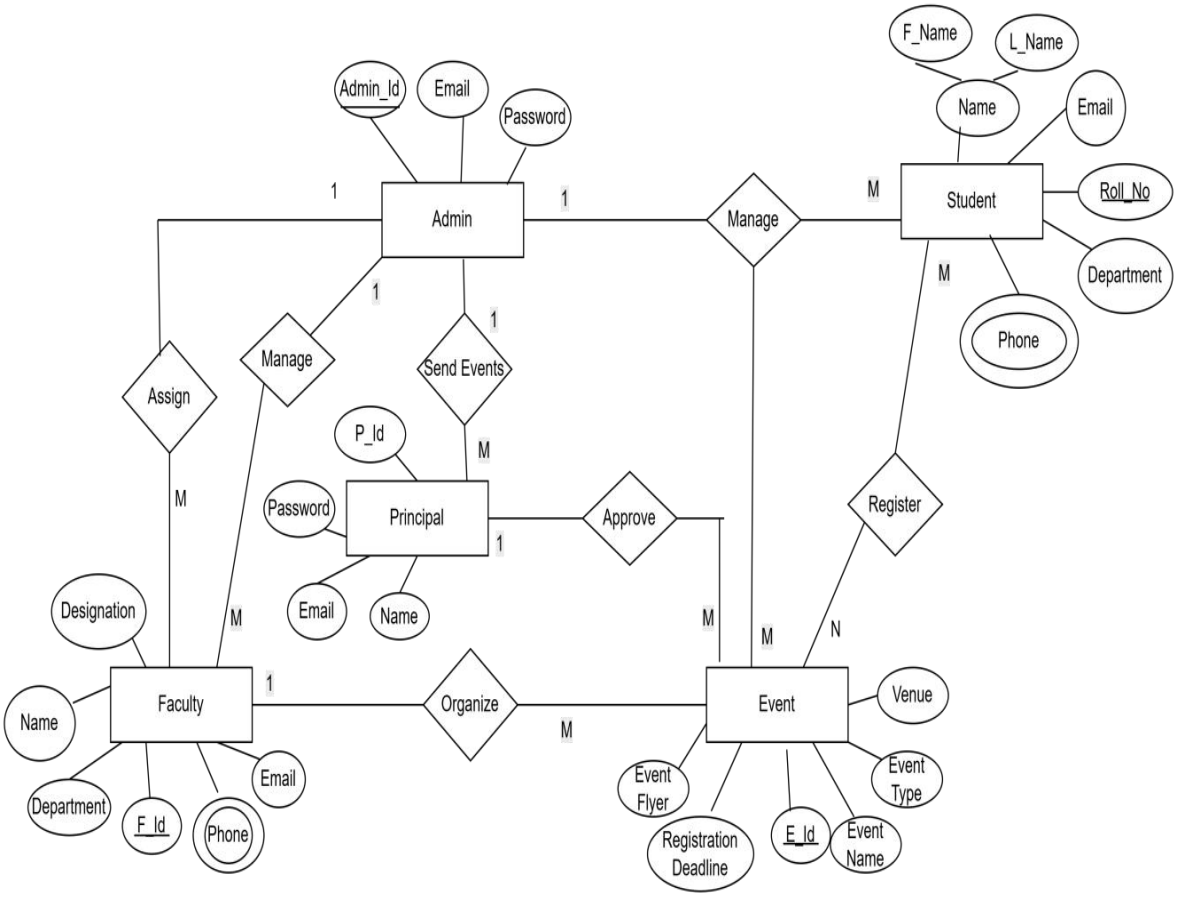


Figure 3.10 ER Diagram

Figure 3.10 illustrates the Entity Relationship Diagram of the College Event Management System. It shows the main entities such as Student, Event, Faculty, Admin and Principal, along with their attributes and relationships.

3.7 DATABASE DESIGN

Database design is the process of organizing data in a structured way so that it can be easily stored, accessed and managed. In the College Event Management System, the database is designed using the Salesforce cloud database. Data is stored using objects, fields and records. Each object represents a table, fields represent columns and records store the actual data. The database stores information such as student details, event details, faculty information, approvals and event registrations. Relationships between objects help connect related data, such as linking students with the events they register for. This structured design helps manage event data efficiently and ensures that information is stored securely and accessed easily when required.

Data Integration

In the College Event Management System, data from different users such as admin, principal, faculty and students is integrated into a single Salesforce platform. The admin creates event details, the principal approves the event and students register for the event. All these details are stored in related objects such as Event, Faculty, Student and Registration. This integration helps maintain all event related information in one system and makes it easier to manage and access data.

Data Independence

In this project, data independence means that changes in one part of the system do not affect other parts. For example, if a new field such as Event Capacity or Event Description is added to the Event object, it does not affect the student registration or approval process. Salesforce allows objects and fields to be modified easily without changing the entire system. This makes the system flexible and easy to maintain.

Data Security

Data security in the College Event Management System is maintained using Salesforce security features. Different users have different access levels. For example, the admin can create and manage events, the principal can approve or reject events and students can only view approved events and register for them. Salesforce uses profiles, roles and permission settings to ensure that only authorized users can access or modify specific data. This helps protect important information and maintain data privacy.

3.8 TABLE DESIGN

Table design is the structure used to store data in a database. Data is organized in the form of tables with rows and columns. Each table represents an entity in the system, and each column represents the attributes of that entity. Table design helps store and manage information in an organized way.

Table 3.1 Student Table

Field Name	Data Type	Description
Roll_No	Number	Unique ID for each student
First_Name	Text	Student first name
Last_Name	Text	Student last name
Email	Email	Student email address
Phone	Phone	Student contact number
Department	Text	Department of the student

Figure 3.1 shows the Student Table, which stores the basic details of students such as roll number, name, email, phone and department. It helps the system identify students and manage event registrations.

Table 3.2 Event Table

Field Name	Data Type	Description
Event_ID	Number	Unique ID for each event
Event_Name	Text	Name of the event
Event_Date	Date	Date of the event
Event_Type	Text	Type of event(Tech/Non-Tech)
Venue	Text	Event location
Event_Flyer	File	Event poster or flyer
Registration_Deadline	Date	Last date to register

Figure 3.2 shows the Event Table, which stores details of events such as event ID, name, date, type, venue, flyer and registration deadline. It helps manage and display event information in the system.

Table 3.3 Faculty Table

Field Name	Data Type	Description
Faculty_ID	Number	Unique ID for faculty
Name	Text	Faculty name
Email	Email	Faculty email
Phone	Phone	Faculty contact number
Department	Text	Faculty department
Designation	Text	Faculty Designation

Figure 3.3 shows the Faculty Table, which stores details of faculty members such as ID, name, email, phone, department and designation. It helps manage faculty information and assign them to organize events.

Table 3.4 Admin Table

Field Name	Data Type	Description
Admin_ID	Number	Unique ID for admin
Email	Email	Admin Email
Password	Text	Admin login password

Figure 3.4 shows the Admin Table, which stores admin details such as admin ID, email and password. It is used to manage system access and perform administrative activities.

Table 3.5 Principal Table

Field Name	Data Type	Description
Principal_ID	Number	Unique ID for principal
Name	Text	Principal name
Email	Email	Principal email
Password	Text	Principal login password

Figure 3.4 shows the Admin Table, which stores admin details such as admin ID, email and password. It is used to manage system access and perform administrative activities.

Table 3.6 Registration Table

Field Name	Data Type	Description
Registration_ID	Number	Unique registration ID
Student_ID	Number	Reference to student
Event_ID	Number	Reference to event
Registration_Date	Date	Data of registration

Figure 3.5 shows the Principal Table, which stores principal details such as ID, name, email and password. It is used for event approval and managing authorization in the system.

SUMMARY

This chapter explains the design of the College Event Management System. The system is divided into modules such as Admin, Principal Approval, Email Notification, Student Registration, Registration Management and Reports & Dashboard. These modules help manage event creation, approval and student registrations. The chapter also includes system diagrams like DFD and Use Case Diagram to show how users interact with the system. It explains input forms for login, event creation and student registration and output screens like dashboards and reports. The chapter also describes the ER diagram, database design and table structure used to store event and registration data in the Salesforce database.

CHAPTER 4

IMPLEMENTATION

4.1 CODE DESCRIPTION

The College Event Management System is developed using the Salesforce platform with the help of objects, flows, approval processes and Lightning components. Custom objects and fields are used to store information such as event details, faculty information, student details and event registrations.

Salesforce Flows are used to automate the process of sending events for approval and sending email notifications to students after the event is approved. The Approval Process allows the principal to approve or reject the event created by the admin.

Lightning components and Experience Cloud pages are used to create the user interface where users can view event details and register for events. Reports and dashboards are also used to display event information and student registration details. These tools help the system manage college events in an organized and efficient way

4.2 STANDARDIZATION OF THE CODING

Standardization of coding means following a common structure and naming method while developing the system. In the College Event Management System, proper standards are maintained while creating objects, fields, flows, components, reports and dashboards. Meaningful names are used for objects and fields so that they are easy to understand and manage. The same format is followed for labels, API names and workflow design.

The system is developed in an organized way so that each module performs a specific function. Proper naming conventions, simple design and clear data relationships help make the system easy to maintain and update. Standardization also improves readability, reduces errors and makes the project easier to understand for future users and developers.

The following configuration snippet demonstrates a standardized automation setup used in the project. It shows how the system checks the event status and sends email notifications to students when the event is approved.

```
IF Event.Status = "Approved"
THEN
   Send Email Notification to Students
   Include Event Name, Date, Venue, and Registration Link
END IF
```

This standardized logic ensures that every approved event automatically sends notifications to students. It maintains consistency in the system workflow and reduces manual work. The same structured approach is followed for event creation, approval processes and student registration automation throughout the project.

4.3 ERROR HANDLING

Error handling is an important part of the system to ensure that the application works smoothly even when unexpected situations occur. In the College Event Management System, proper validation and system checks are used to prevent errors and provide clear messages to users.

When users enter incorrect or incomplete information, the system displays validation messages to guide them. For example, if the admin tries to create an event without entering required details such as event name or date, the system shows an error message and prevents the record from being saved. Similarly, if a student attempts to register for an event without filling all required fields, the system will not allow the submission until the correct information is provided.

The system also handles errors during processes such as event approval, email notification and student registration. If a process fails, the system records the issue and prevents incorrect data from being stored. Salesforce validation rules, flows and permission settings help maintain data accuracy and prevent unauthorized actions.

CHAPTER 5

TESTING AND RESULT

5.1 TESTING

Testing is used to check whether the College Event Management System works correctly and produces the expected results. Different types of testing are performed to verify the accuracy, reliability and usability of the system.

5.1.1 Unit Testing

Unit testing is used to test each module separately. It checks whether one individual function works correctly.

CASE 1

Module: Event Creation Module
Test Type: Unit Testing
Input: Admin enters event details and clicks save
Output: Event record created successfully

TEST
Input: Event Name, Date, Venue, Department, Faculty Organizer
Output: Event saved in the system
Analysis: The event creation module works correctly as a single unit.
Result: Pass

5.1.2 Integration Testing

Integration testing is used to check whether two or more modules work together properly.

CASE 2

Module: Event Approval + Email Notification Module
Test Type: Integration Testing

Input: Principal approves the event

Output: Event status changes to Approved and email is sent to students

TEST

Input: Principal approval action

Output: Approved status updated and event email sent

Analysis: The approval module and email notification module work together correctly.

Result: Pass

5.1.3 System Testing

System testing is used to verify the complete working of the full system.

CASE 3

Module: Complete Event Management Process

Test Type: System Testing

Input: Admin creates event, principal approves, student registers

Output: Entire process completed successfully

TEST

Input: Event creation → approval → email → student registration

Output: Event created, approved, emailed, and registration completed successfully

Analysis: The complete workflow of the system works correctly from beginning to end.

Result: Pass

5.1.4 Security Testing

Security testing is used to verify that only authorized users can access specific functions and data.

CASE 4

Module: Principal Approval Module

Test Type: Security Testing

Input: Student tries to access the approval page

Output: Access denied

TEST

Input: Student login attempts to open event approval page

Output: Unauthorized access blocked

Analysis: The system protects restricted pages from unauthorized users.

Result: Pass

5.1.5 Validation Testing

Validation testing is used to check whether the data entered into the system is correct and complete.

CASE 5

Module: Student Registration Form

Test Type: Validation Testing

Input: Student submits the form without required details

Output: Error message displayed and form not submitted

TEST

Input: Incomplete registration form

Output: Validation error shown

Analysis: The system prevents incomplete or invalid data from being saved.

Result: Pass

5.1.6 User Acceptance Testing

User acceptance testing is used to verify whether the system is easy to use and satisfies user requirements.

CASE 6

Module: Student Registration Module

Test Type: User Acceptance Testing

Input: Student opens event link and completes registration

Output: Student registers successfully without difficulty

TEST

Input: Registration link and student details

Output: Registration completed successfully

Analysis: The system is simple, clear, and easy for users to operate.

Result: Pass

SUMMARY

This chapter explains the testing performed on the College Event Management System to ensure that the system works correctly and produces the expected results. Different types of testing such as unit testing, integration testing, system testing, security testing, validation testing and user acceptance testing are carried out. These tests verify that individual modules work properly, different modules interact correctly, the complete system functions as expected and only authorized users can access specific features. The testing also ensures that correct data is entered into the system and that the system is easy for users to operate. The results show that all modules work successfully and the system performs reliably.

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 CONCLUSION

The College Event Management System can be improved further by adding more advanced features in the future. One possible enhancement is the introduction of a mobile friendly interface or mobile application, so that students and faculty members can easily access event details and register using their smartphones. This will improve accessibility and make the system more convenient to use.

Another enhancement is the addition of QR code based event registration, where students can scan a QR code on the event flyer and register directly. The system can also be extended to include attendance management, so that faculty members can mark the attendance of students who actually participate in the event. This will help maintain both registration and attendance records in one system.

The project can also be improved by adding automatic certificate generation for students who participate in events. Another useful feature would be real time notifications through SMS or mobile alerts in addition to email notifications. Advanced analytics and dashboards can also be added to provide deeper insights into student participation, popular events and department wise involvement.In the future, the system can be integrated with other college management modules to create a more complete campus management solution. These enhancements will make the system more powerful, efficient and beneficial for the institution.

6.2 FUTURE ENHANCEMENT

The College Event Management System can be improved further by adding more advanced features in the future.One possible enhancement is attendance management, where faculty members can mark the attendance of students who actually participate in the event. This will help the college maintain both registration and attendance records in the same system. Another useful feature is automatic certificate generation, where students who successfully attend an event can receive certificates directly through the system.

The project can also be enhanced by adding real-time notifications such as SMS alerts or app notifications in addition to email. This will help students receive event updates more quickly. Another improvement is the addition of advanced analytics and dashboards to provide detailed insights such as most popular events, department wise participation trends and student engagement statistics.

In the future, the system can also be integrated with other academic modules to create a more complete college management solution. These enhancements will improve the functionality, efficiency and usefulness of the system.

APPENDICES

A. SAMPLE CODING

UPCOMING EVENTS

EventController.cls

```
public with sharing class EventController {
   @AuraEnabled(cacheable=true)
   public static List<Event__c> getUpcomingApprovedEvents() {
      Date today = Date.today();
      try {
         List<Event__c> events = [
            SELECT Id, Name, Event_Date__c, Venue__c, Event_Type__c,
Event_Flyer_URL__c
            FROM Event__c
            WHERE Status__c = 'Approved'
              AND Event_Date__c >= :today
            ORDER BY Event_Date__c ASC
            LIMIT 20
         ];
         return events;
      } catch (Exception e) {
         throw new AuraHandledException('Unable to load upcoming approved events.');
      }
   }
   @AuraEnabled(cacheable=true)
   public static List<Event__c> getAllEvents() {
      try {
         List<Event__c> events = [
            SELECT Id, Name, Status__c, Venue__c, Event_Date__c
            FROM Event__c
            ORDER BY Event_Date__c DESC, Name
```

```
        ];
        return events;
    } catch (Exception e) {
        throw new AuraHandledException('Unable to load events.');
    }
}
@AuraEnabled
public static void updateEvent(Id eventId, String name, String location, String status) {
    try {
        Event__c evt = [SELECT Id FROM Event__c WHERE Id = :eventId LIMIT 1];
        if (name != null) {
            evt.Name = name;
        }
        if (location != null) {
            evt.Location__c = location;
        }
        if (status != null) {
            evt.Status__c = status;
        }
        update evt;
    } catch (Exception e) {
        throw new AuraHandledException('Unable to update event: ' + e.getMessage());
    }
@AuraEnabled
public static Id createEvent(
    String name,
    String eventType,
    String description,
    Date eventDate,
    Date registrationDeadline,
    String location,
    String venue,
    Integer capacity,
```

```
        String status,
        String eventFlyerUrl,
        Id facultyOrganizerId,
        String participatingDepartment
    ) {
        try {
            Event__c evt = new Event__c();
            evt.Name = name;
            if (eventType != null) { evt.Event_Type__c = eventType; }
            if (description != null) { evt.Description__c = description; }
            if (eventDate != null) { evt.Event_Date__c = eventDate; }
            if (registrationDeadline != null) { evt.Registration_Deadline__c = registrationDeadline; }
            if (location != null) { evt.Location__c = location; }
            if (venue != null) { evt.Venue__c = venue; }
            if (capacity != null) { evt.Capacity__c = capacity; }
            if (status != null) { evt.Status__c = status; }
            if (eventFlyerUrl != null) { evt.Event_Flyer_URL__c = eventFlyerUrl; }
            if (facultyOrganizerId != null) { evt.Faculty_Organizer__c = facultyOrganizerId; }
            if (participatingDepartment != null) { evt.Participating_Department__c = participatingDepartment; }
            insert evt;
            return evt.Id;
        } catch (Exception e) {
            throw new AuraHandledException('Unable to create event: ' + e.getMessage());
        }
    }
    @AuraEnabled(cacheable=true)
    public static List<Faculty__c> getFacultyList() {
        try {
            return [SELECT Id, Name FROM Faculty__c ORDER BY Name ASC LIMIT 200];
        } catch (Exception e) {
            throw new AuraHandledException('Unable to load faculty list.');
```

```
        }
    }
    @AuraEnabled(cacheable=true)
    public static Integer getRegistrationCountForEvent(Id eventId) {
        try {
            if (eventId == null) {
                throw new AuraHandledException('Missing eventId.');
            }
            // Aggregate to count registrations by Event__c
            // Added explicit field reference check and better error handling
            try {
                List<AggregateResult> agg = [
                    SELECT COUNT(Id) cnt
                    FROM Registration__c
                    WHERE Event__c = :eventId
                ];
                if (!agg.isEmpty()) {
                    Object v = agg[0].get('cnt');
                    return (v instanceof Integer) ? (Integer)v : Integer.valueOf(String.valueOf(v));
                }
            } catch (Exception e) {
                // Enhanced error handling to provide more context
                System.debug('Error in getRegistrationCountForEvent: ' + e.getMessage());
                throw new AuraHandledException('Unable to load registration count: ' +
e.getMessage());
            }
            return 0;
        } catch (Exception e) {
            throw new AuraHandledException('Unable to load registration count.');
        }
    }
}
```

UpcomingApprovedEvent.html

```
<template>
  <section class="slds-card slds-m-around_medium">
    <div class="slds-card__header slds-grid">
      <header class="slds-media slds-media_center slds-has-flexi-truncate">
        <div class="slds-media__figure">
          <span class="slds-icon_container slds-icon-standard-event" title="Upcoming Approved Events">
            <svg class="slds-icon slds-icon_small" aria-hidden="true">
              <use xlink:href="/_slds/icons/standard-sprite/svg/symbols.svg#event"></use>
            </svg>
          </span>
        </div>
        <div class="slds-media__body">
          <h2 class="slds-card__header-title">
            <span class="slds-text-heading_small">Upcoming Approved Events</span>
          </h2>
        </div>
      </header>
    </div>

    <div class="slds-card__body slds-card__body_inner">
      <template if:true={loading}>
        <div class="slds-is-relative" style="min-height: 4rem;">
          <lightning-spinner alternative-text="Loading events" size="small"></lightning-spinner>
        </div>
      </template>

      <template if:true={error}>
        <div class="slds-notify slds-notify_alert slds-theme_alert-texture slds-theme_error" role="alert">
          <span class="slds-assistive-text">Error</span>
```

```
      <h2>{errorMessage}</h2>
    </div>
  </template>

  <template if:false={loading}>
    <template if:true={hasEvents}>
      <ul class="slds-grid slds-wrap slds-gutters">
        <template for:each={events} for:item="evt">
          <li key={evt.id} class="slds-col slds-size_1-of-1">
            <article class="slds-card slds-m-around_x-small event-card">
              <div class="slds-card__header slds-grid">
                <header class="slds-media slds-media_center slds-has-flexi-truncate">
                  <div class="slds-media__body">
                    <h3class="slds-card__header-titleslds-truncate"
title={evt.name}>{evt.name}</h3>
                  </div>
                </header>
              </div>
              <div class="slds-card__body slds-card__body_inner">
                <p class="slds-text-color_weak slds-m-bottom_x-small">
                  <lightning-icon  icon-name="utility:event"  size="x-small"  class="slds-m-
right_x-small"></lightning-icon>
                  <span title={evt.dateDisplay}>{evt.dateDisplay}</span>
                </p>
                <template if:true={evt.venue}>
                  <p class="slds-text-color_weak slds-m-bottom_x-small">
                    <lightning-icon icon-name="utility:location" size="x-small" class="slds-m-
right_x-small"></lightning-icon>
                    <span class="slds-truncate" title={evt.venue}>{evt.venue}</span>
                  </p>
                </template>
                <template if:true={evt.type}>
                  <p class="slds-text-color_weak slds-m-bottom_x-small">
```

```
                <lightning-icon icon-name="utility:tags" size="x-small" class="slds-m-right_x-small"></lightning-icon>
                <span class="slds-truncate" title={evt.type}>{evt.type}</span>
              </p>
            </template>
            <template if:true={evt.registrationCount}>
              <p class="slds-text-color_weak slds-m-bottom_x-small">
                <lightning-icon icon-name="utility:users" size="x-small" class="slds-m-right_x-small"></lightning-icon>
                <span>Registrations: {evt.registrationCount}</span>
              </p>
            </template>
            <template if:true={evt.loadingCount}>
              <p class="slds-text-color_weak slds-m-bottom_x-small">
                <lightning-spinner alternative-text="Loading registrations" size="x-small" class="slds-m-right_x-small"></lightning-spinner>
                <span>Loading registrations...</span>
              </p>
            </template>
            <template if:true={evt.flyerUrl}>
              <p class="slds-m-top_x-small">
                <lightning-button
                  variant="base"
                  label="View Flyer"
                  onclick={handleFlyerClick}
                  data-url={evt.flyerUrl}
                  class="slds-p-left_none">
                </lightning-button>
              </p>
            </template>
          </div>
        </article>
      </li>
```

```
          </template>
        </ul>
      </template>
      <template if:false={hasEvents}>
        <div class="slds-p-around_medium slds-text-align_center slds-text-color_weak">
          No upcoming approved events.
        </div>
      </template>
    </template>
  </div>
 </section>
</template>
```

UpcomingApprovedEvent.js

```
import { LightningElement, wire } from 'lwc';
importgetUpcomingApprovedEventsfrom
'@salesforce/apex/EventController.getUpcomingApprovedEvents';
importgetRegistrationCountForEventfrom
'@salesforce/apex/EventController.getRegistrationCountForEvent';

export default class UpcomingApprovedEvents extends LightningElement {
   events = [];
   loading = true;
   error;
   selectedEventId;
   selectedEventCount;
   counting = false;

   @wire(getUpcomingApprovedEvents)
   wiredEvents({ error, data }) {
      if (data) {
         // Map defensively in case fields are missing and include additional fields
         this.events = (data || []).map((evt) => {
```

```
      const dateVal = evt && evt.Event_Date__c ? new Date(evt.Event_Date__c) : null;
        const dateDisplay = dateVal
          ? dateVal.toLocaleDateString(undefined, {
              year: 'numeric',
              month: 'short',
              day: '2-digit'
            })
          : '';
        return {
          id: evt && evt.Id,
          name: (evt && evt.Name) || '',
          dateDisplay,
          venue: (evt && evt.Venue__c) || '',
          type: (evt && evt.Event_Type__c) || '',
          flyerUrl: (evt && evt.Event_Flyer_URL__c) || '',
          registrationCount: undefined,
          loadingCount: false
        };
    });
    this.error = undefined;
    this.loading = false;
    return;
  }
  if (error) {
    // Normalize Apex/AuraHandledException or network errors
    this.error =
      (error && error.body && error.body.message) ||
      (typeof error === 'string' ? error : JSON.stringify(error));
    this.loading = false;
    return;
  }
  // Fallback: no data and no error
  this.loading = false;
```

```
}
handleFlyerClick(event) {
  const url = event.currentTarget?.dataset?.url;
  if (url) {
    // Open in a new tab; rely on browser to block if disallowed
    window.open(url, '_blank', 'noopener,noreferrer');
  }
}
async handleCardClick(event) {
  // Event card container click handler (delegate from article)
  const id = event.currentTarget?.dataset?.id;
  if (!id) return;
  // Find the event in our events array
  const eventIndex = this.events.findIndex(evt => evt.id === id);
  if (eventIndex === -1) return;
  this.selectedEventId = id;
  this.selectedEventCount = undefined;
  this.counting = true;

  try {
    // Set loading state
    this.events[eventIndex].loadingCount = true;
    this.events[eventIndex].registrationCount = undefined;

    const count = await getRegistrationCountForEvent({ eventId: id });
    const parsedCount = typeof count === 'number' ? count : parseInt(count, 10);

    // Update the event object with registration count
    this.events[eventIndex].registrationCount = parsedCount;
    this.selectedEventCount = parsedCount;
  } catch (e) {
    // surface a friendly message; keep component resilient
    this.error =
```

```
            (e && e.body && e.body.message) ||
            (typeof e === 'string' ? e : JSON.stringify(e));
        } finally {
            this.events[eventIndex].loadingCount = false;
            this.counting = false;
        }
    }

    get hasEvents() {
        return Array.isArray(this.events) && this.events.length > 0;
    }

    get errorMessage() {
        return this.error || '';
    }

    get hasSelection() {
        return !!this.selectedEventId;
    }
}
```

UpcomingApprovedEvent.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>65.0</apiVersion>
    <isExposed>true</isExposed>
    <masterLabel>Upcoming Approved Events</masterLabel>
    <description>Displays upcoming Event__c records with Status__c = "Approved" and future Event_Date__c and Registration_Deadline__c.</description>
    <targets>
        <target>lightningCommunity__Page</target>
        <target>lightningCommunity__Default</target>
        <target>lightning__HomePage</target>
```

```
        <target>lightning__AppPage</target>
        <target>lightning__RecordPage</target>
        <target>lightning__Tab</target>
    </targets>
    <targetConfigs>
        <targetConfig targets="lightning__RecordPage">
            <objects>
                <object>Event__c</object>
            </objects>
        </targetConfig>
    </targetConfigs>
</LightningComponentBundle>
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta http-equiv="Content-type" content="text/html; charset=UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Event Registration</title>
<link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&dis
play=swap" rel="stylesheet">
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }

  body {
    font-family: 'Poppins', sans-serif;
    background: #0f172a;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
```

```
  position: relative;
}

body::before {
  content: '';
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  background:url('https://images.unsplash.com/photo-1540575467063-
178a50c2df87?auto=format&fit=crop&w=1920&q=80') no-repeat center center;
  background-size: cover;
  opacity: 0.25;
  z-index: -1;
}

.container {
  background: rgba(255, 237, 213, 0.95);
  width: 100%;
  max-width: 500px;
  margin: 30px 16px;
  padding: 36px 32px 30px;
  border-radius: 20px;
  box-shadow: 0 20px 60px rgba(0,0,0,0.3);
  position: relative;
  overflow: hidden;
}

.container::before {
  content: '';
  position: absolute;
  top: 0; left: 0; right: 0;
  height: 5px;
  background: linear-gradient(90deg, #1a73e8, #4fc3f7, #1a73e8);
```

```
}

h2 {
  text-align: center;
  color: #1a73e8;
  font-size: 24px;
  font-weight: 700;
  margin-bottom: 22px;
}

label {
  display: block;
  margin-top: 12px;
  margin-bottom: 5px;
  font-weight: 600;
  font-size: 13px;
  color: #334155;
}

input, select {
  width: 100%;
  padding: 10px 12px;
  margin-bottom: 6px;
  border: 1.5px solid #cbd5e1;
  border-radius: 8px;
  font-family: 'Poppins', sans-serif;
  font-size: 14px;
  color: #1e293b;
  background: #fff;
  transition: border-color 0.2s, box-shadow 0.2s;
  outline: none;
}
```

```
input:focus, select:focus {
  border-color: #1a73e8;
  box-shadow: 0 0 0 3px rgba(26, 115, 232, 0.15);
}

button {
  width: 100%;
  padding: 13px;
  margin-top: 18px;
  background: linear-gradient(135deg, #1a73e8, #4a90f4);
  color: #fff;
  border: none;
  border-radius: 10px;
  font-family: 'Poppins', sans-serif;
  font-weight: 600;
  font-size: 15px;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  cursor: pointer;
  box-shadow: 0 4px 18px rgba(26, 115, 232, 0.35);
  transition: all 0.3s ease;
}

button:hover {
  background: linear-gradient(135deg, #155ab6, #1a73e8);
  box-shadow: 0 6px 24px rgba(26, 115, 232, 0.45);
  transform: translateY(-2px);
}

@media (max-width: 480px) {
  .container { padding: 28px 18px 22px; }
  h2 { font-size: 20px; }
}
```

```
</style>
</head>
<body>

<div class="container">
  <h2>Event Registration</h2>

  <form   action="https://webto.salesforce.com/servlet/servlet.WebToLead?encoding=UTF-
8&orgId=00Dfj00000JKU9G" method="POST">

    <input type="hidden" name="oid" value="00Dfj00000JKU9G">
    <inputtype="hidden"name="retURL"
value="https://gokila0.github.io/EventHunt/thankyou.html">

    <!-- Lead required company -->
    <input type="hidden" name="company" value="Student">

    <label for="first_name">First Name</label>
    <input id="first_name" name="first_name" maxlength="40" type="text" required>

    <label for="last_name">Last Name</label>
    <input id="last_name" name="last_name" maxlength="80" type="text" required>

    <label for="00Nfj00003CWRJB">Event Name</label>
    <input     id="00Nfj00003CWRJB"     name="00Nfj00003CWRJB"     maxlength="250"
type="text" required>

    <label for="00Nfj00003CWR33">Roll Number</label>
    <input     id="00Nfj00003CWR33"     name="00Nfj00003CWR33"     maxlength="100"
type="text" required>

    <label for="00Nfj00003CWRMP">Department</label>
```

```
<input id="00Nfj00003CWRMP" name="00Nfj00003CWRMP" maxlength="100" type="text" required>

<label for="phone">Phone</label>
<input id="phone" name="phone" maxlength="40" type="text" required>

<label for="email">Email</label>
<input id="email" name="email" maxlength="80" type="email" required>

<label for="00Nfj00003CWUTx">Student Year</label>
<select id="00Nfj00003CWUTx" name="00Nfj00003CWUTx" required>
  <option value="">--Select Year--</option>
  <option value="UG-I">UG-I</option>
  <option value="UG-II">UG-II</option>
  <option value="UG-III">UG-III</option>
  <option value="UG-IV">UG-IV</option>
  <option value="PG-I">PG-I</option>
  <option value="PG-II">PG-II</option>
</select>

<label for="00Nfj00003CWY69">Gender</label>
<select id="00Nfj00003CWY69" name="00Nfj00003CWY69" required>
  <option value="">--Select Gender--</option>
  <option value="MALE">MALE</option>
  <option value="FEMALE">FEMALE</option>
</select>

<label for="00Nfj00003CWXur">Registration Date</label>
<input id="00Nfj00003CWXur" name="00Nfj00003CWXur" type="text" readonly required>

<button type="submit">Register</button>
</form>
```

```
</div>

<script>
  const regDateField = document.getElementById("00Nfj00003CWXur");
  const today = new Date();
  const yyyy = today.getFullYear();
  const mm = String(today.getMonth() + 1).padStart(2, '0');
  const dd = String(today.getDate()).padStart(2, '0');
  regDateField.value = `${yyyy}-${mm}-${dd}`;
</script>

</body>
</html>
```

Thankyou.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Successful</title>
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&dis
play=swap" rel="stylesheet">
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body {
      font-family: 'Poppins', sans-serif;
      background: #0f172a;
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      position: relative;
```

```
}
body::before {
  content: '';
  position: fixed;
  top: 0; left: 0;
  width: 100%; height: 100%;
  background:url('https://images.unsplash.com/photo-1540575467063-
178a50c2df87?auto=format&fit=crop&w=1920&q=80') no-repeat center center;
  background-size: cover;
  opacity: 0.25;
  z-index: -1;
}
.card {
  background: rgba(255, 237, 213, 0.95);
  padding: 50px 40px;
  border-radius: 20px;
  box-shadow: 0 20px 60px rgba(0,0,0,0.3);
  text-align: center;
  max-width: 480px;
  width: 100%;
  margin: 20px;
  position: relative;
  overflow: hidden;
}
.card::before {
  content: '';
  position: absolute;
  top: 0; left: 0; right: 0;
  height: 5px;
  background: linear-gradient(90deg, #1a73e8, #4fc3f7, #1a73e8);
}
.icon { font-size: 64px; margin-bottom: 16px; }
```

```
    p {
      color: #475569;
      font-size: 15px;
      line-height: 1.6;
      margin-bottom: 24px;
    }
    {
      display: inline-block;
      padding: 12px 32px;
      background: linear-gradient(135deg, #1a73e8, #4a90f4);
      color: #fff;
      text-decoration: none;
      border-radius: 10px;
      font-weight: 600;
      font-size: 14px;
      box-shadow: 0 4px 18px rgba(26, 115, 232, 0.35);
    }
    a:hover {
      background: linear-gradient(135deg, #155ab6, #1a73e8);
      box-shadow: 0 6px 24px rgba(26, 115, 232, 0.45);
      transform: translateY(-2px);
    }
  </style>
</head>
<body>
  <div class="card">
    <div class="icon">   </div>
    <h2>Registration Successful!</h2>
    <p>Thank you for registering for the event. Your details have been submitted successfully. You will receive a confirmation soon.</p>
  </div>
</body>
</html>
```

B. SCREENSHOTS

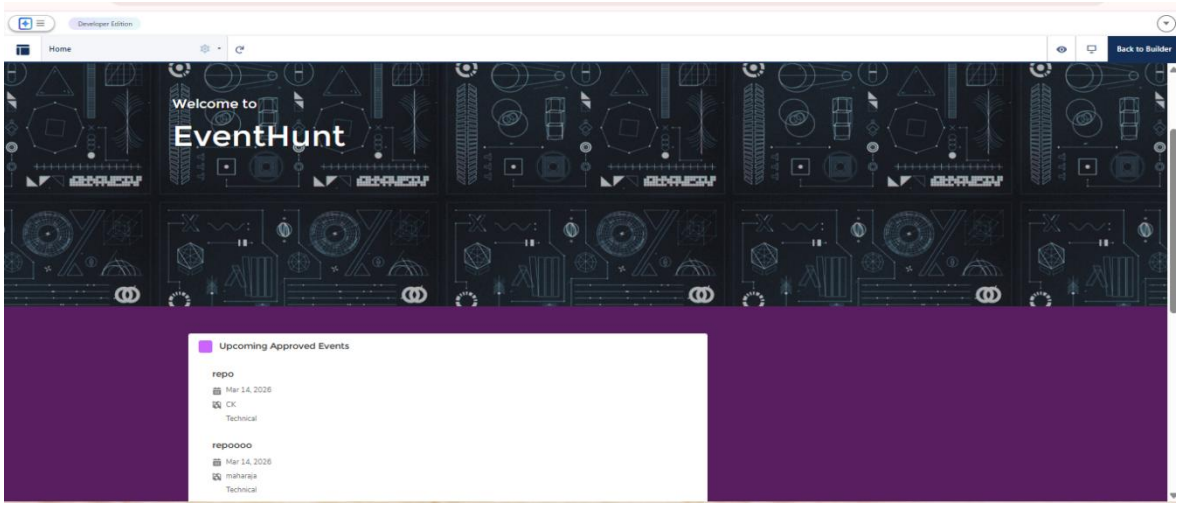


Figure B.1 Home Page

In Figure B.1 This is the home page of the college event management system, welcoming users after logging into the application.

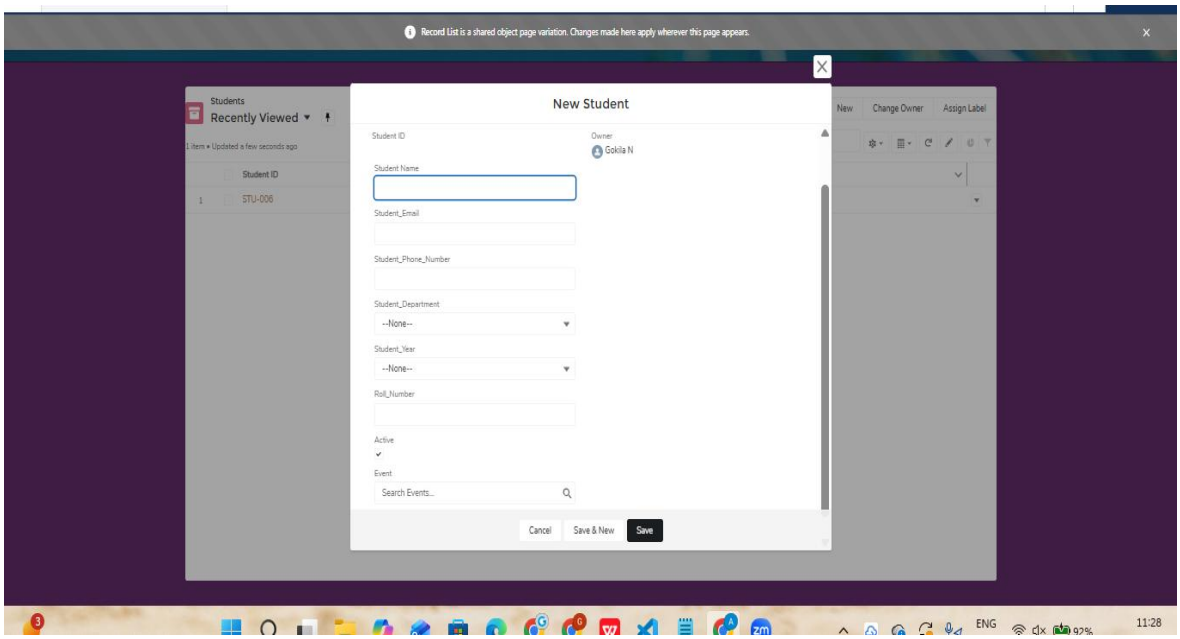


Figure B.2 Event Creation

Figure B.2 shows the Event Creation page, where the admin or faculty member can create a new event. The form includes fields such as event name, event type, date, venue, registration deadline, and event description. The user enters all required details and submits the form to create the event.

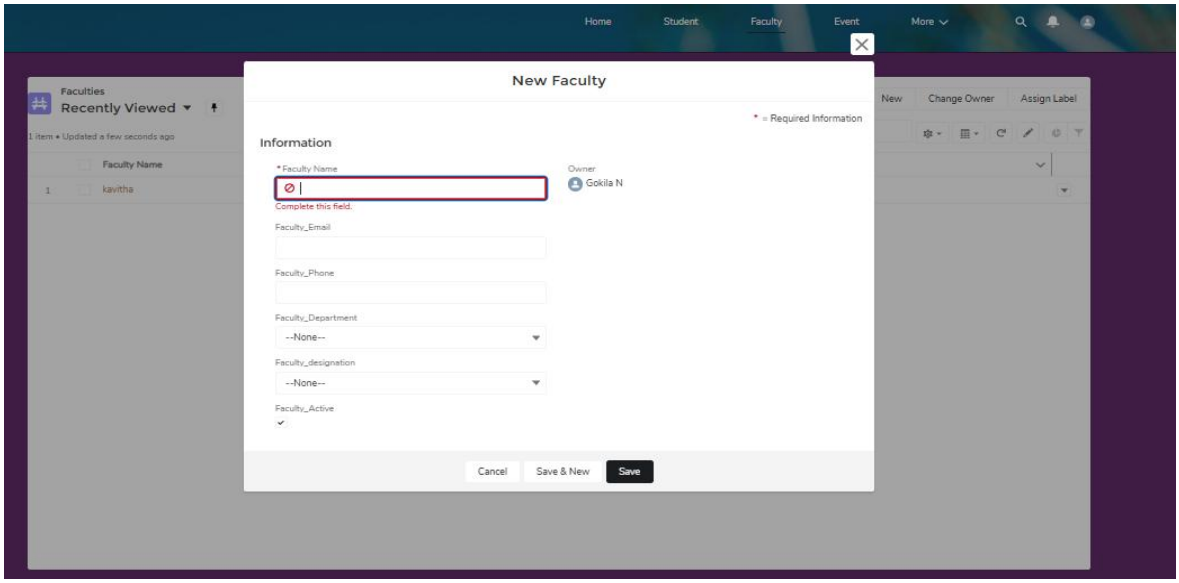


Figure B.3 New Faculty Form

Figure B.3 shows the New Faculty Form, which is used to add a new faculty member to the system. The form includes fields such as faculty name, email, phone number, department, designation, and active status.

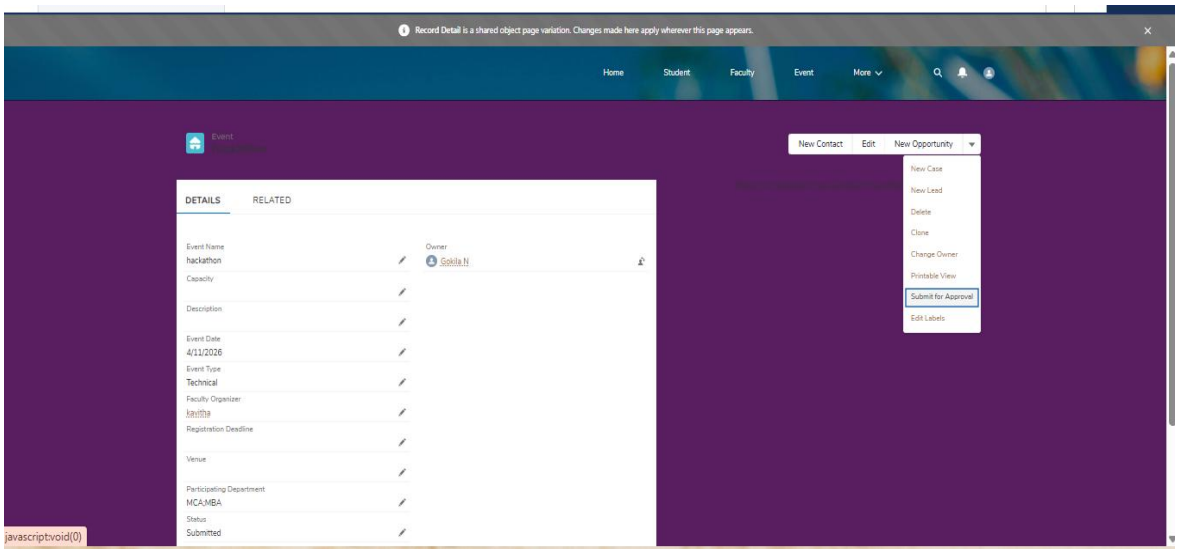


Figure B.4 Submit for Approval

Figure B.5 shows the notification received by the Principal when an event is submitted for approval. The system automatically sends a notification with event details such as event name, date, and organizer.

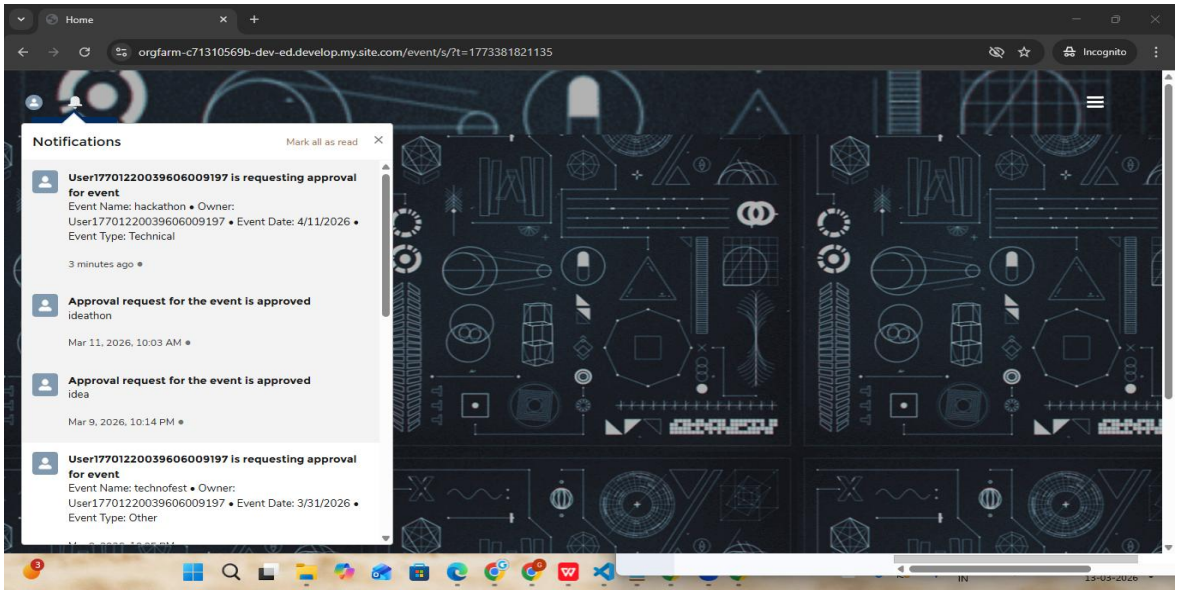


Figure B.5 Principal get notification

Figure B.5 shows the notification received by the Principal when an event is submitted for approval. The system automatically sends a notification with event details such as event name, date, and organizer.

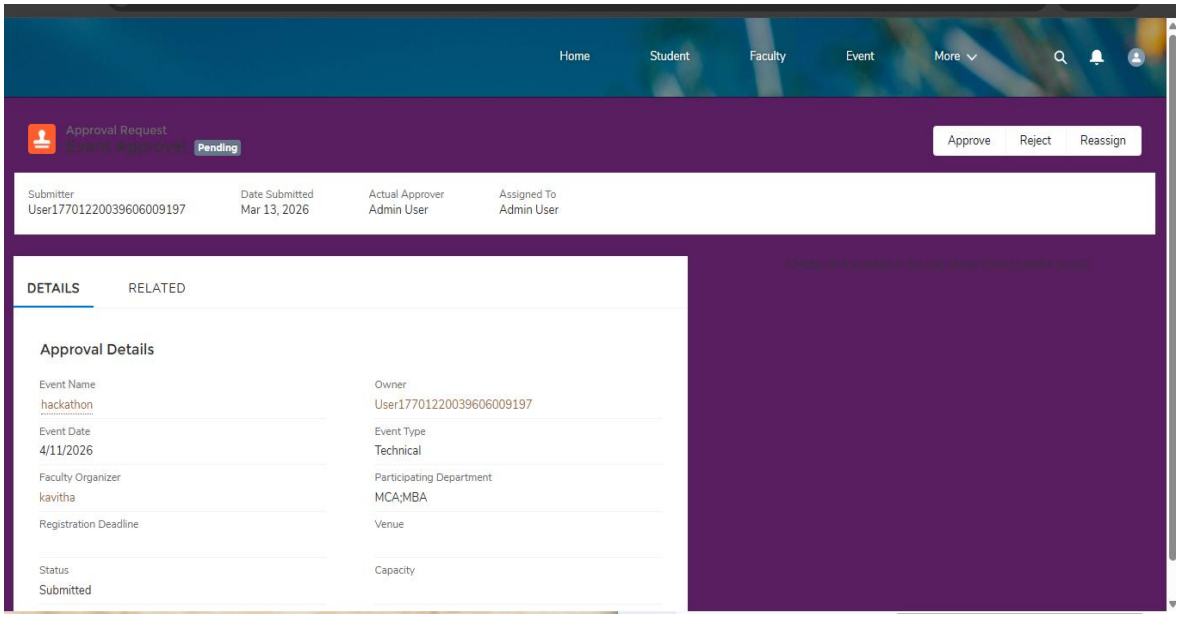


Figure B.6 Approve or Reject the event

Figure B.6 shows the approval screen used by the Principal to review and take action on the event request. The principal can view the complete event details such as event name, date, venue, and organizer before making a decision.

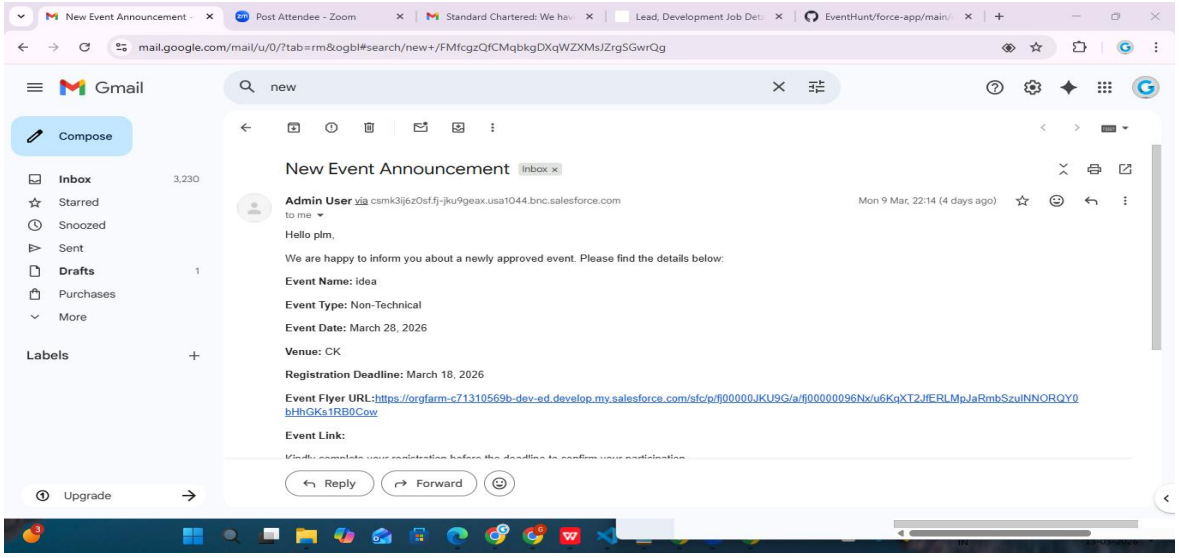


Figure B.7 Student get the email

Figure B.7 shows the email notification received by students after the event is approved by the principal. The email contains important details such as event name, date, venue, and a registration link

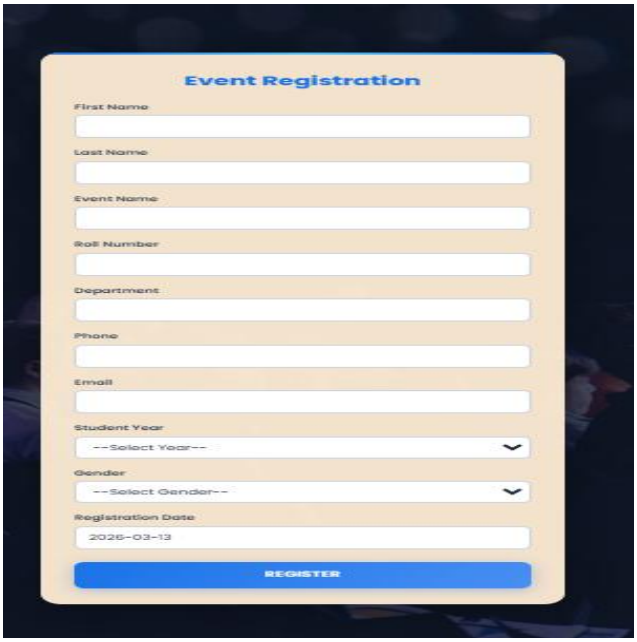


Figure B.8 Registration Form

Figure B.8 shows the Event Registration Form used by students to register for an event. The form includes fields such as student name, email, department, and phone number.

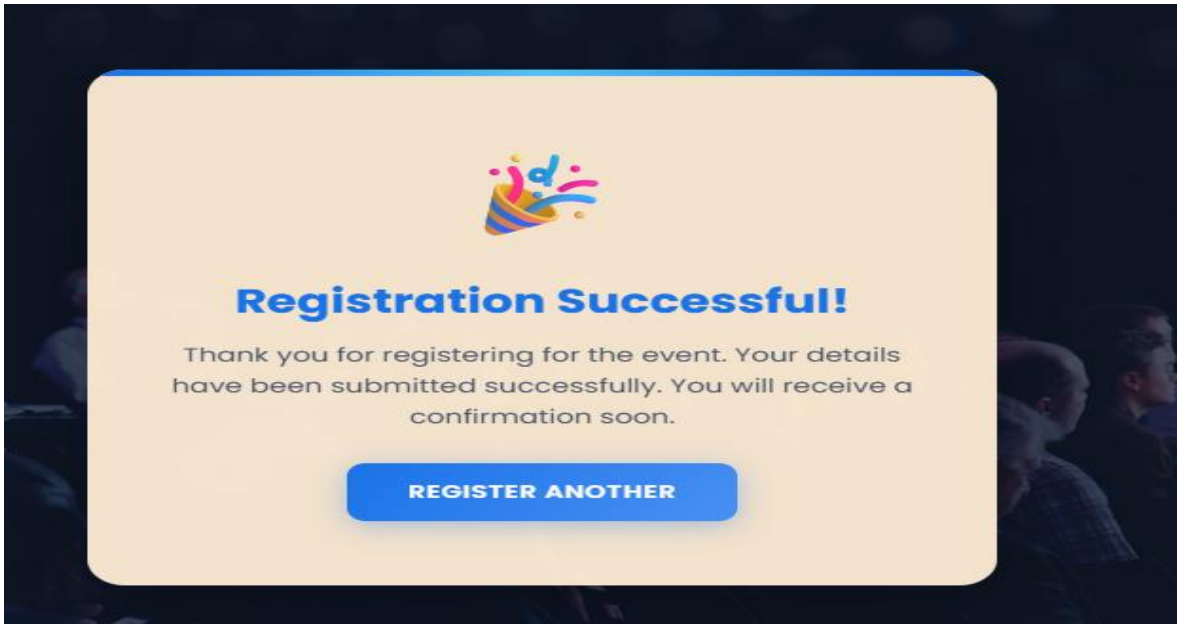
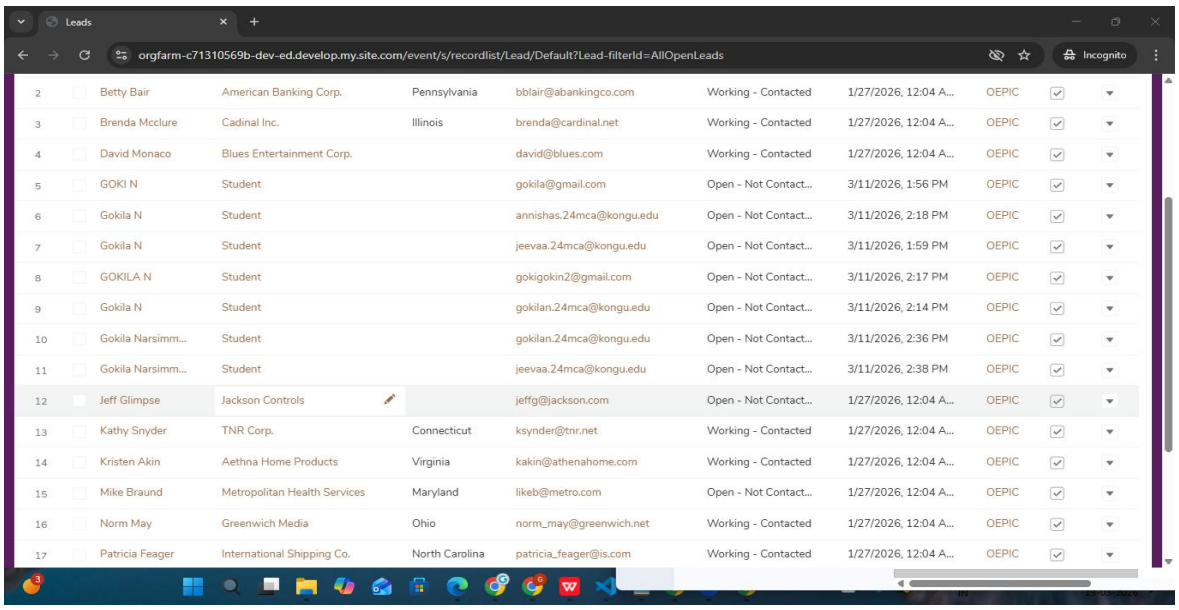


Figure B.9 Successfully Register the event page

Figure B.9 shows the confirmation page displayed after a student successfully registers for an event. The page confirms that the registration has been completed and may display details such as event name and registration status.



2	Betty Bair	American Banking Corp.	Pennsylvania	bblair@abankingco.com	Working - Contacted	1/27/2026, 12:04 A...	OEPIC	
3	Brenda Mcclure	Cadinal Inc.	Illinois	brenda@cardinal.net	Working - Contacted	1/27/2026, 12:04 A...	OEPIC	
4	David Monaco	Blues Entertainment Corp.		david@blues.com	Working - Contacted	1/27/2026, 12:04 A...	OEPIC	
5	GOKI N	Student		gokila@gmail.com	Open - Not Contact...	3/11/2026, 1:56 PM	OEPIC	
6	Gokila N	Student		annishas.24mca@kongu.edu	Open - Not Contact...	3/11/2026, 2:18 PM	OEPIC	
7	Gokila N	Student		jeevaa.24mca@kongu.edu	Open - Not Contact...	3/11/2026, 1:59 PM	OEPIC	
8	GOKILA N	Student		gokigokin2@gmail.com	Open - Not Contact...	3/11/2026, 2:17 PM	OEPIC	
9	Gokila N	Student		gokilan.24mca@kongu.edu	Open - Not Contact...	3/11/2026, 2:14 PM	OEPIC	
10	Gokila Narsimm...	Student		gokilan.24mca@kongu.edu	Open - Not Contact...	3/11/2026, 2:36 PM	OEPIC	
11	Gokila Narsimm...	Student		jeevaa.24mca@kongu.edu	Open - Not Contact...	3/11/2026, 2:38 PM	OEPIC	
12	Jeff Glimpse	Jackson Controls		jeffg@jackson.com	Open - Not Contact...	1/27/2026, 12:04 A...	OEPIC	
13	Kathy Snyder	TNR Corp.	Connecticut	ksynder@tnr.net	Working - Contacted	1/27/2026, 12:04 A...	OEPIC	
14	Kristen Akin	Aethna Home Products	Virginia	kakin@athenahome.com	Working - Contacted	1/27/2026, 12:04 A...	OEPIC	
15	Mike Braund	Metropolitan Health Services	Maryland	likeb@metro.com	Open - Not Contact...	1/27/2026, 12:04 A...	OEPIC	
16	Norm May	Greenwich Media	Ohio	norm_may@greenwich.net	Working - Contacted	1/27/2026, 12:04 A...	OEPIC	
17	Patricia Feager	International Shipping Co.	North Carolina	patricia_feager@is.com	Working - Contacted	1/27/2026, 12:04 A...	OEPIC	

Figure B.10 Student data store in salesforce

Figure B.7 shows the email notification received by students after the event is approved by the principal. The email contains important details such as event name, date, venue, and a registration link.

REFERENCES

BOOK & JOURNAL REFERENCES

[1] David Liu, *Beginning Salesforce Development*, Apress Media, 2019.

[2] Paul Goodey, *Salesforce CRM – The Definitive Admin Handbook*, Packt Publishing, 2020.

[3] Sharif Shaalan, "Salesforce Platform Enterprise Architecture", *International Journal of Cloud Computing*, Vol. 9, No. 2, 2020.

[4] A. Kumar and S. Patel, "Cloud-Based Customer Relationship Management Systems Using Salesforce", *International Journal of Computer Science and Engineering*, Vol. 10, No. 4, 2022.

[5] Mohith Shrivastava, *Learning Salesforce Development with Apex*, Packt Publishing, 2018.

[6] J. Smith, "Implementation of Cloud-Based CRM Systems in Educational Institutions", *International Journal of Advanced Research*, Vol. 11, No. 3, 2021.

[7] Andrew Fawcett, *Salesforce Platform Enterprise Architecture*, Packt Publishing, 2018.

[8] R. Gupta and P. Sharma, "Cloud Computing Applications for Student Management Systems", *International Journal of Information Technology*, Vol. 13, No. 2, 2021.

WEB REFERENCES

[1] https://www.salesforce.com

[2] https://developer.salesforce.com

[3] https://trailhead.salesforce.com

[4] https://help.salesforce.com

[5] https://developer.mozilla.org